Text as Data Workshop

SCISS Sharjah 2026

Siefken

2026-05-25

Contents

Setup

Load Packages

```
install.packages("pacman")
```

```
library(pacman)

p_load(tidyverse,
       quanteda,
       quanteda.textmodels,
       quanteda.textplots,
       quanteda.textstats,
       stm,
       devtools,
       SnowballC)
```

Install & Load Additional Packages from GitHub

Download Presidential Debate Dataset from Github (Martherus 2020; https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3611815)

```
devtools::install_github("jamesmartherus/debates")
devtools::install_github("kbenoit/quanteda.dictionaries")
devtools::install_github("quanteda/quanteda.sentiment")
```

```
library("quanteda.sentiment")
library("quanteda.dictionaries")
library(debates)
```

Load the Data

```
data(debate_transcripts)
```

Data Pre-Processing

```
debates <- debate_transcripts %>%
  filter(candidate == 1) %>% #Only include candidates
  filter(election_year >= 2000, type == "Pres") %>% #Limit the data
  group_by(speaker, date) %>%
  summarise(text = paste(text, collapse = " "), #Merge all speeches per candidate and day into one text
            election_year = first(election_year), #Needed to keep the meta data
            type = first(type),
            .groups = "drop") %>%
  mutate(doc_id = row_number())
```

Create Corpus

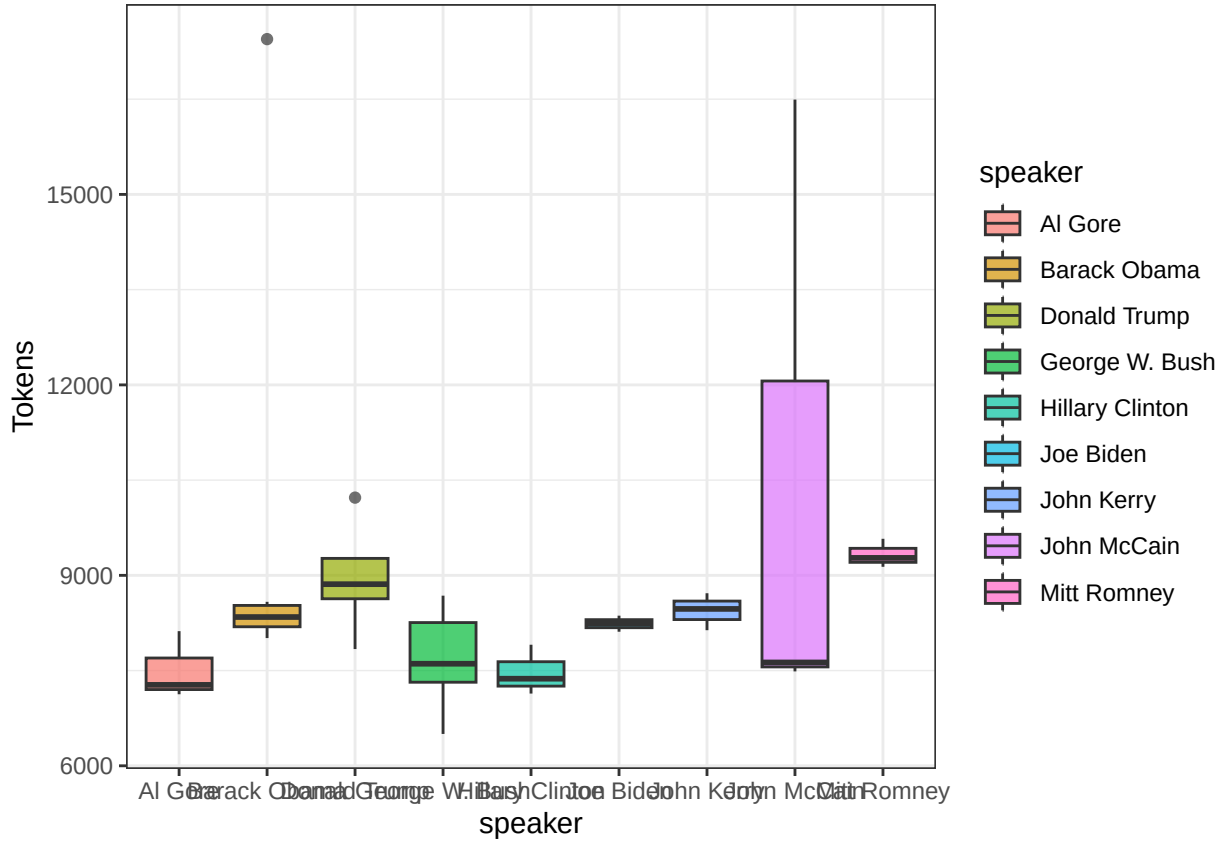
```
corpus_debate <- corpus(debates,
                        text_field = "text",
                        docid_field = "doc_id")
```

Overview of the Data

```
summary_debate <- summary(corpus_debate, n = ndoc(corpus_debate)) #By default, summary only shows the f
```

```
View(summary_debate)
```

```
summary_debate %>%
  ggplot(aes(x = speaker, y = Tokens)) +
  geom_boxplot(aes(fill = speaker), alpha = .7) +
  theme_bw()
```



Subsetting the Corpus

```
corpus_obama <- corpus_subset(corpus_debate,
                              speaker == "Barack Obama")
```

Document Feature Matrix

Turn dataset into a DFM, required for e.g. Dictionary Analysis.

```
dfm_debate <- corpus_debate %>%
  tokens(
    remove_punct = TRUE,
    remove_numbers = TRUE,
    split_hyphens = TRUE
  ) %>%
  tokens_wordstem(
    language = quanteda_options("language_stemmer"),
    verbose = quanteda_options("verbose")
  ) %>%
  tokens_remove(c("example1", "example2", stopwords("en"))) %>% #Remove specified words
  dfm()
```

Trim Infrequent Features

```
dfm_debate_trim <- dfm_trim(dfm_debate,
                            min_docfreq = 2, #Has to appear in at least two texts
                            min_termfreq = 5) #Has to appear at least 5 times total

nfeat(dfm_debate_trim) #Number of unique words in the data
```

```
## [1] 2359
```

Inspect the DFM

```
View(convert(dfm_debate, to = "data.frame"))
```

Analyse the DFM

Top 20 Features (Overall)

```
top20_feat <- textstat_frequency(dfm_debate_trim, n = 20)
```

```
View(top20_feat)
```

Top 20 Features by Speaker

```
docvars(dfm_debate_trim)
```

```
##            speaker       date election_year type
## 1          Al Gore 2000-10-03          2000 Pres
## 2          Al Gore 2000-10-11          2000 Pres
## 3          Al Gore 2000-10-17          2000 Pres
## 4     Barack Obama 2008-09-26          2008 Pres
## 5     Barack Obama 2008-10-07          2008 Pres
## 6     Barack Obama 2008-10-15          2008 Pres
## 7     Barack Obama 2012-10-03          2012 Pres
```

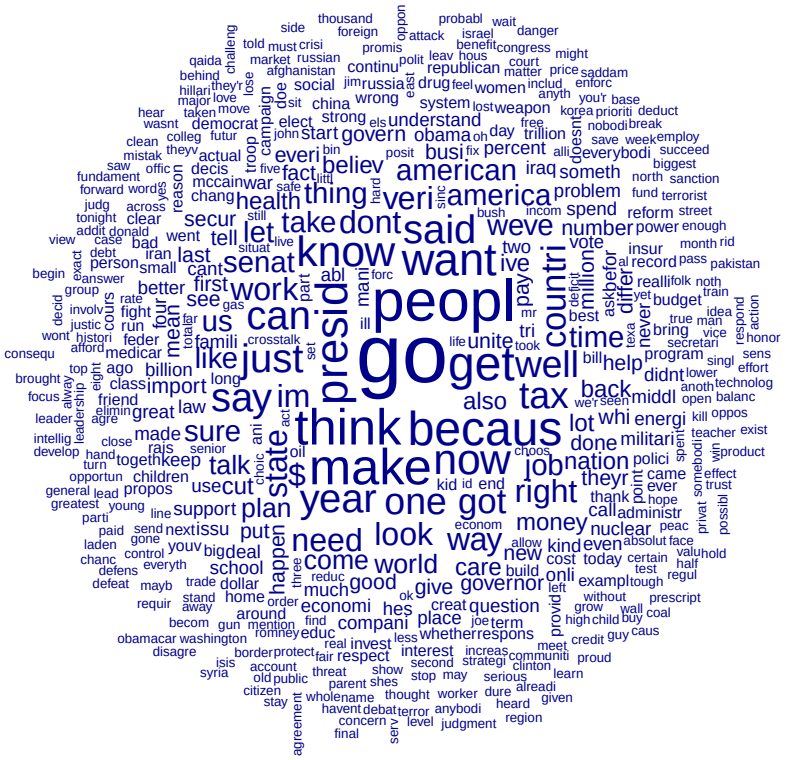
```
## 8     Barack Obama 2012-10-16          2012 Pres
## 9     Barack Obama 2012-10-22          2012 Pres
## 10    Donald Trump 2016-09-26          2016 Pres
## 11    Donald Trump 2016-10-09          2016 Pres
## 12    Donald Trump 2016-10-19          2016 Pres
## 13    Donald Trump 2020-09-29          2020 Pres
## 14    Donald Trump 2020-10-22          2020 Pres
## 15  George W. Bush 2000-10-03          2000 Pres
## 16  George W. Bush 2000-10-11          2000 Pres
## 17  George W. Bush 2000-10-17          2000 Pres
## 18  George W. Bush 2004-09-30          2004 Pres
## 19  George W. Bush 2004-10-08          2004 Pres
## 20  George W. Bush 2004-10-13          2004 Pres
## 21 Hillary Clinton 2016-09-26          2016 Pres
## 22 Hillary Clinton 2016-10-09          2016 Pres
## 23 Hillary Clinton 2016-10-19          2016 Pres
## 24       Joe Biden 2020-09-29          2020 Pres
## 25       Joe Biden 2020-10-22          2020 Pres
## 26      John Kerry 2004-09-30          2004 Pres
## 27      John Kerry 2004-10-08          2004 Pres
## 28      John Kerry 2004-10-13          2004 Pres
## 29     John McCain 2008-09-26          2008 Pres
## 30     John McCain 2008-10-07          2008 Pres
## 31     John McCain 2008-10-15          2008 Pres
## 32     Mitt Romney 2012-10-03          2012 Pres
## 33     Mitt Romney 2012-10-16          2012 Pres
## 34     Mitt Romney 2012-10-22          2012 Pres
```

```
top20_feat_president <- textstat_frequency(dfm_debate_trim,
                                n = 20,
                                group = docvars(dfm_debate_trim, "speaker"))
```

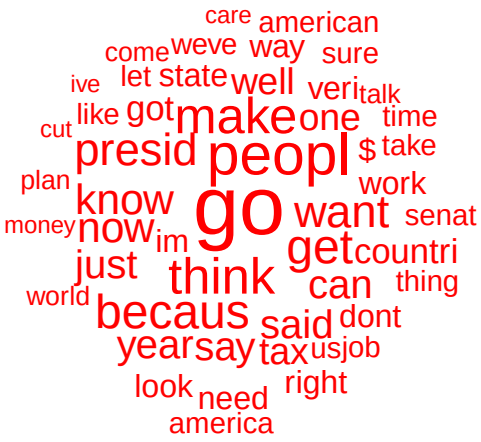
```
View(top20_feat_president)
```

Wordclouds

```
textplot_wordcloud(dfm_debate_trim)
```



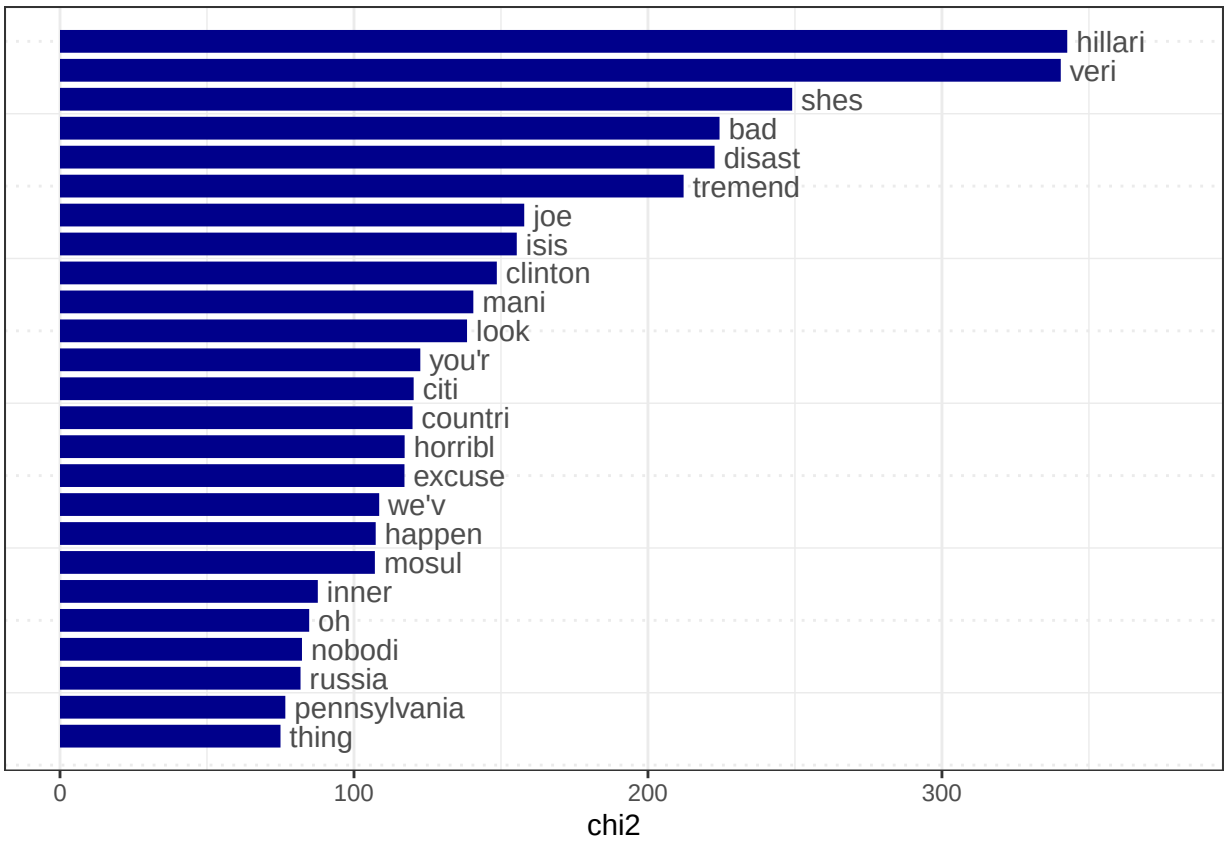

```
textplot_wordcloud(dfm_debate_trim,
                   max_words = 50,
                   color = "red",
                   rotation = 0)
```



Keyness Analysis

Keyness analysis determines which words are used more by a predetermined group than by the other groups. E.g. which words are used more by Trump than other candidates.

```
#Lets group the DFM by Candidates
dfm_debate_grouped <- dfm_debate_trim %>%
  dfm_group(groups = docvars(dfm_debate_trim, "speaker"))

keyness_trump <- textstat_keyness(dfm_debate_grouped, target = "Donald Trump")

textplot_keyness(keyness_trump,
                 show_reference = FALSE,
                 n = 25)
```



Dictionary Analysis

Create a Custom Dictionary

Note: As we used stemming on our data, the dictionary should also contain stemmed words.

```
wordStem("integration", language = "en")
```

```
## [1] "integr"
```

```
list_dict <- list(
  economy = c("economi", "job", "employ", "gdp", "growth", "budget", "deficit", "debt"),
  healthcare = c("health", "medicaid", "medicar", "insur", "hospit", "doctor", "patient"),
  education = c("educ", "school", "teacher", "student", "colleg", "univers", "tuition"),
  environment = c("climat", "environ", "energi", " green", "emiss", "renew", "pollut"),
  immigration = c("immigr", "border", "citizen", "visa", "migrant", "deport"),
  foreign_policy = c("foreign", "diplomat", "diplomaci", "allianc", "nato", "trade", "tariff", "sanctio
  social_security = c("retir", "pension", "senior", "elder", "social secur"),
  crime = c("crime", "crimin", "polic", "justic", "prison", "law")
)

topic_dict <- dictionary(list_dict)
```

Apply the Dictionary

```
topic_counts <- dfm_lookup(dfm_debate_trim,
                           topic_dict)

df_topic_counts <- convert(topic_counts,
                           to = "data.frame")

df_topic_counts <- cbind(docvars(dfm_debate_trim), df_topic_counts) #Merge meta info back on the datase
```

Summarise by Speaker

```
df_topic_counts %>%
  group_by(speaker) %>%
  summarise(mean_immigration = mean(immigration),
            mean_economy = mean(economy),
            mean_environment = mean(environment),
            mean_crime = mean(crime))
```

```
## # A tibble: 9 x 5
##   speaker         mean_immigration mean_economy mean_environment mean_crime
##   <chr>                      <dbl>        <dbl>            <dbl>      <dbl>
## 1 Al Gore                     1            22.7            10         17
## 2 Barack Obama                3.83         51.8            16.5        5
## 3 Donald Trump                9.4          23.4             5.4       21
## 4 George W. Bush              8            16.2             4.5       17.2
## 5 Hillary Clinton            12.7          38.7             7.33      18.3
## 6 Joe Biden                   1.5          29              18         20
## 7 John Kerry                  5.33         33.3             2.67       9.33
## 8 John McCain                 5.67         32.3            13.3        7.33
## 9 Mitt Romney                 4.67         82              15.3        3
```

Sentiment Analysis

Sentiment Dictionary from the `quanteda.sentiment` package (Benoit, https://github.com/quanteda/quanteda.sentiment).

For this example, we will use the Lexicoder Sentiment Dictionary by Young & Soroka (2012).

```
View(data_dictionary_LSD2015)
View(list_dict)
```

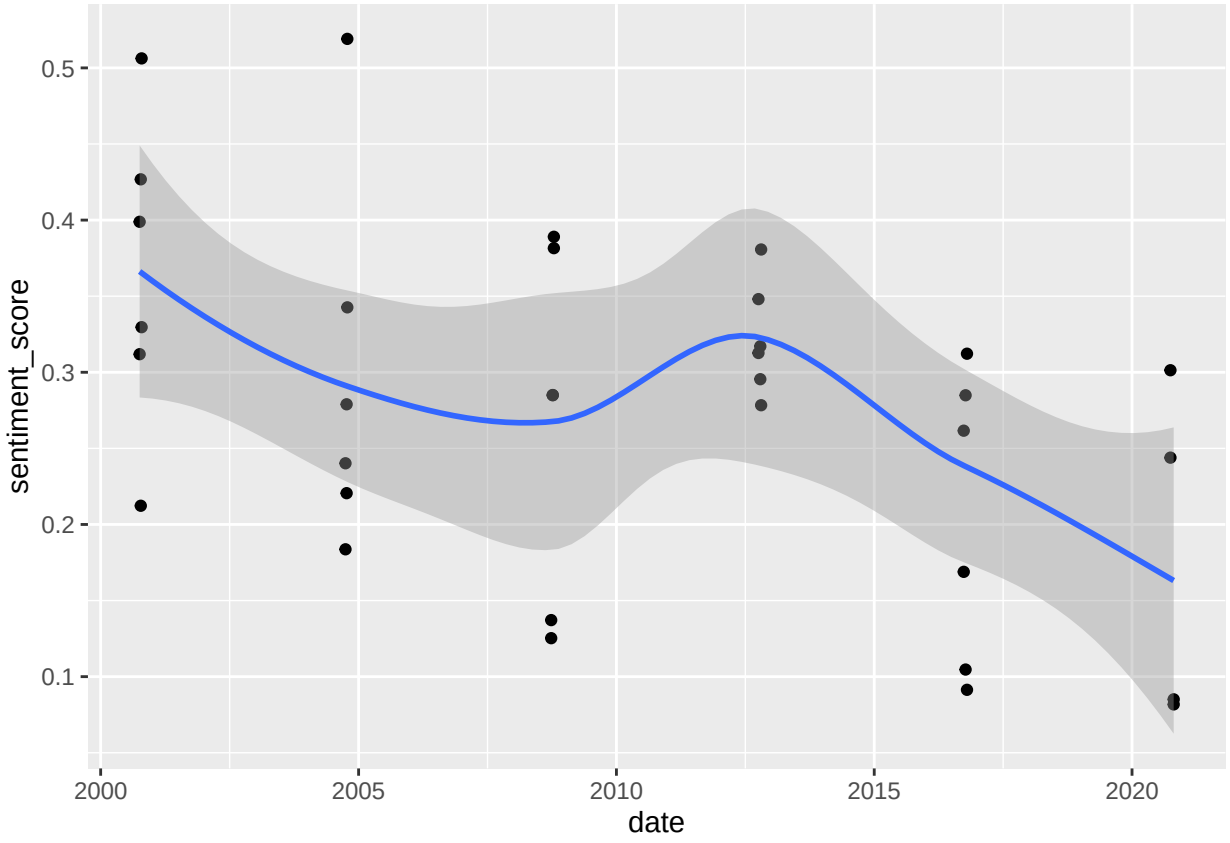
```
debate_sent <- dfm_lookup(dfm_debate_trim,
                          data_dictionary_LSD2015)

debate_sent_df <- convert(debate_sent,
                          to = "data.frame")

debate_sent_df <- cbind(docvars(dfm_debate_trim),
                        debate_sent_df)

debate_sent_df <- debate_sent_df %>%
  mutate(sentiment_score = (positive - negative)/(positive+negative),
```

```
        perc_pos = positive/(positive+negative),
        perc_neg = negative/(positive+negative))
```

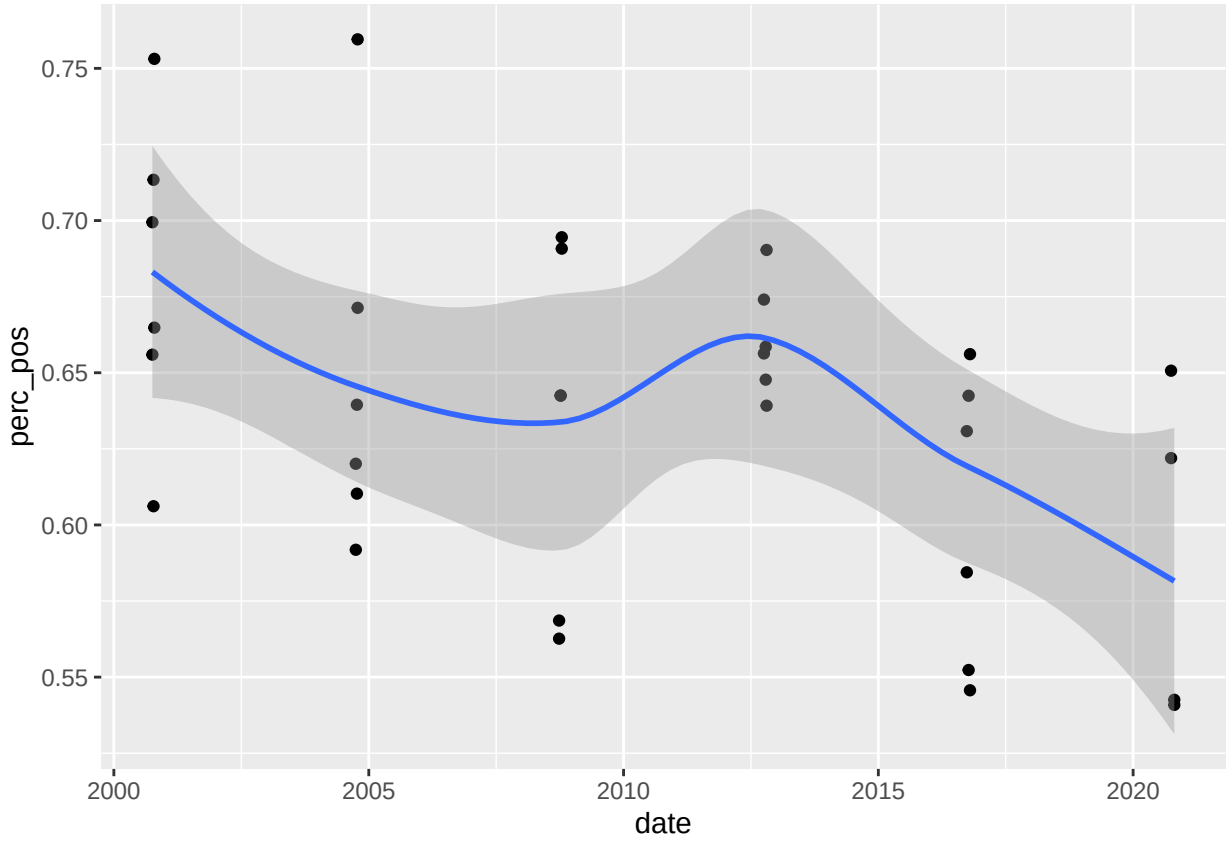
```
View(debate_sent_df)
```

Visualise Sentiment Over Time

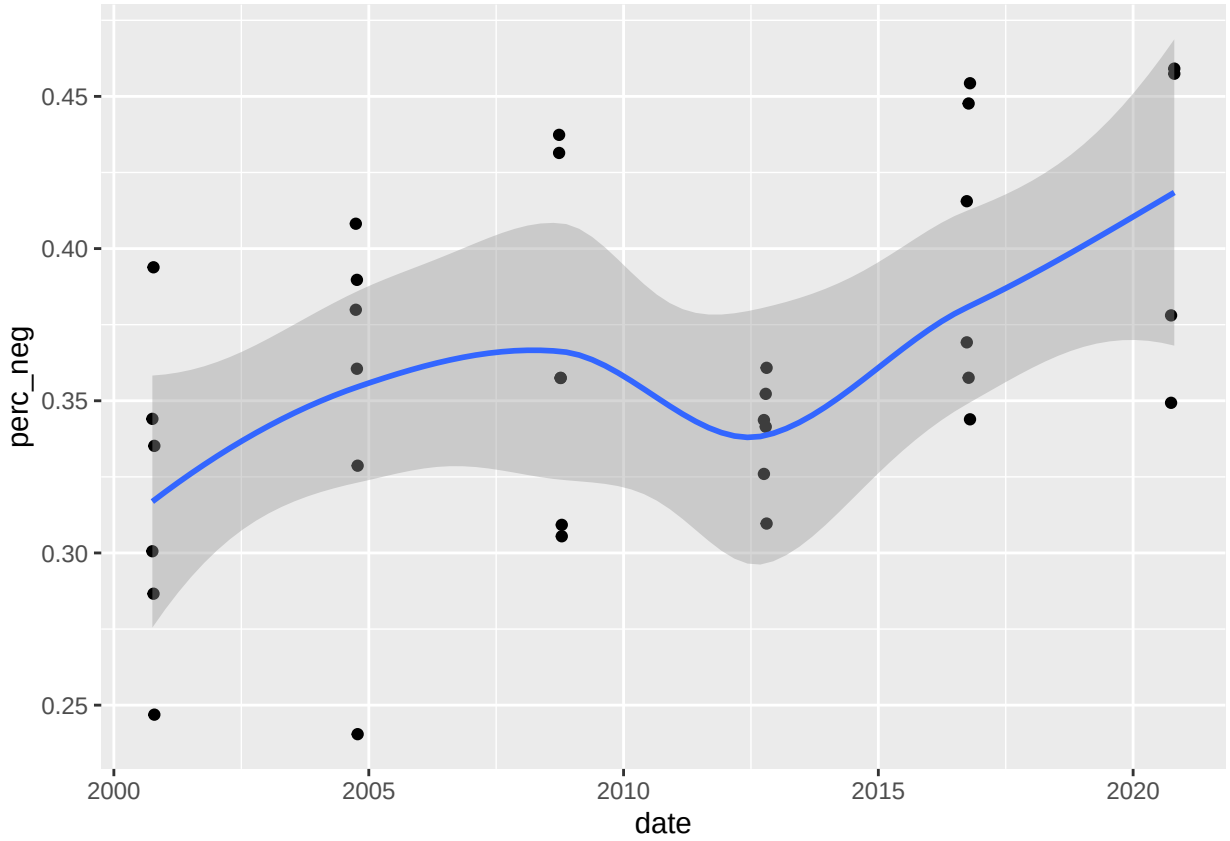
```
ggplot(debate_sent_df, aes(x = date, y = sentiment_score)) +
  geom_point() +
  geom_smooth()
```




```
ggplot(debate_sent_df, aes(x = date, y = perc_pos)) +
  geom_point() +
  geom_smooth()
```




```
ggplot(debate_sent_df, aes(x = date, y = perc_neg)) +
  geom_point() +
  geom_smooth()
```



Topic Models

For this, we will use the STM package (Roberts 2025, https://cran.r-project.org/web/packages/stm/index.html).

```
dfm_stm <- convert(dfm_debate_trim, to = "stm", docvars(dfm_debate_trim))
```

Fit the STM Model

Now we specify how many topics we expect within the data by defining `K`. We tell STM to run only 50 iterations max — skip this argument in a real analysis.

```
stm_model <- stm(documents = dfm_stm$documents,
                 vocab = dfm_stm$vocab,
                 K = 6,
                 prevalence = ~ stm::s(election_year), #We allow the prevalence of topics to vary by ye
                 max.em.its = 50,
                 data = dfm_stm$meta,
                 seed = 26052026)
```

Inspect Topics

Common rule: Look at the FREX words as the most frequent and exclusive words per topic.

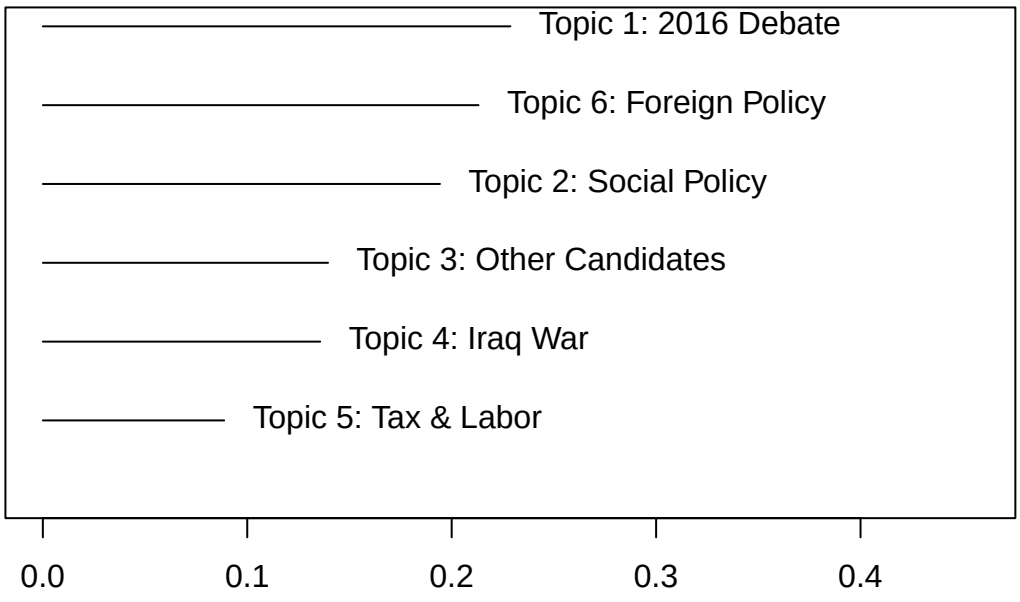
```
labelTopics(stm_model)
```

```
## Topic 1 Top Words:
##       Highest Prob: go, peopl, becaus, veri, know, say, look
##       FREX: we'r, they'r, you'r, isis, hillari, donald, we'v
##       Lift: audit, blumenth, burisma, cage, crazi, depreci, fauci
##       Score: we'r, hillari, they'r, you'r, solicit, we'v, tremend
## Topic 2 Top Words:
##       Highest Prob: go, peopl, get, presid, tax, make, job
##       FREX: incom, lower, deficit, small, rate, medicar, insur
##       Lift: bowl, dodd, proof, reelect, simpson, jeremi, grab
##       Score: reelect, candi, romney, dodd, obamacar, proof, percent
## Topic 3 Top Words:
##       Highest Prob: go, make, weve, got, becaus, think, now
##       FREX: mccain, romney, john, iran, weve, invest, pakistan
##       Lift: 20th, muddl, refocus, resurg, tighten, centrifug, crippl
##       Score: romney, refocus, pakistan, mccain, region, governor, afghanistan
## Topic 4 Top Words:
##       Highest Prob: presid, go, think, peopl, world, america, war
##       FREX: saddam, hussein, oppon, weapon, terrorist, war, iraq
##       Lift: 11th, armor, blair, casualti, reconstruct, steadfast, transform
##       Score: disarm, saddam, hussein, oppon, mass, destruct, summit
## Topic 5 Top Words:
##       Highest Prob: senat, obama, go, know, want, state, can
##       FREX: obama, senat, georgia, petraeus, russian, fought, strategi
##       Lift: acquir, colombia, waziristan, wider, benefici, freddi, greed
##       Score: obama, colombia, petraeus, pork, precondit, earmark, acquir
## Topic 6 Top Words:
##       Highest Prob: think, peopl, go, make, get, want, one
##       FREX: texa, school, children, senior, gun, prescript, vice
##       Lift: aftermath, cast, entrust, imf, ingenu, instant, jame
##       Score: aftermath, texa, serbia, milosev, profil, vice, farmer
```

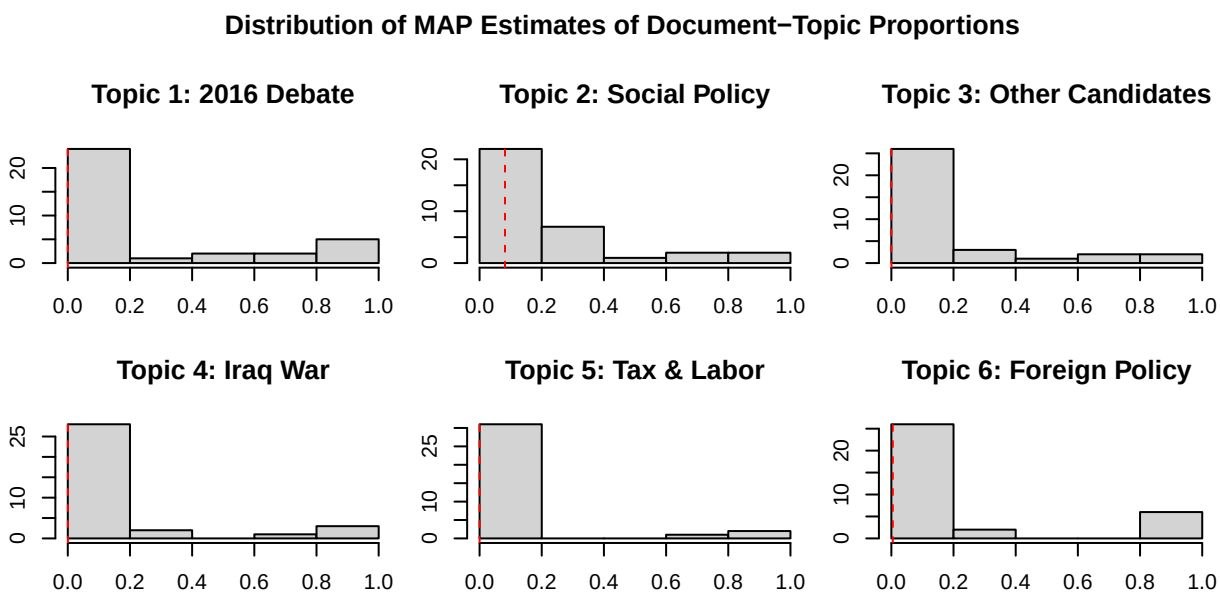
```
topic_labels <- c("2016 Debate",
                  "Social Policy",
                  "Other Candidates",
                  "Iraq War",
                  "Tax & Labor",
                  "Foreign Policy")
```

Plot Topic Model Results

```
plot(stm_model, type = "summary",
     labeltype = "frex",
     custom.labels = topic_labels)
```

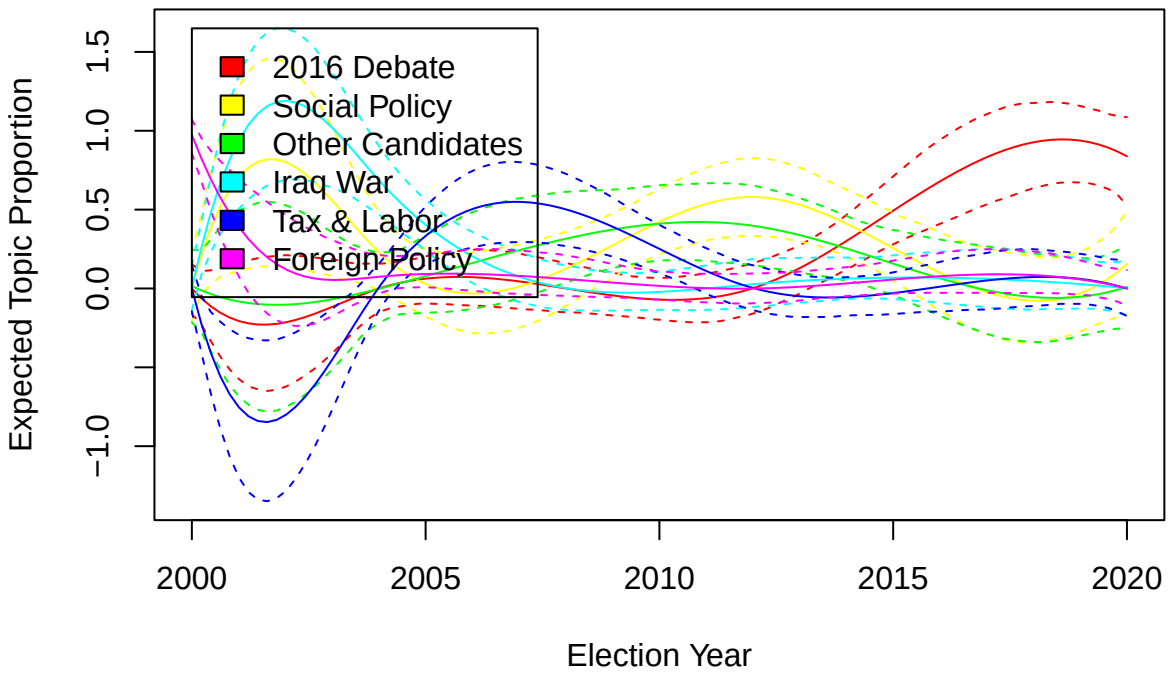



```
plot(stm_model, type = "hist",
     labeltype = "frex",
     custom.labels = topic_labels)
```



Estimate Effects

```
prep <- estimateEffect(~ stm::s(election_year),
                       stm_model,
                       meta = dfm_stm$meta)

par(mfrow = c(1,1))
plot(prep, covariate = "election_year",
     topics = 1:6,
     model = stm_model,
     method = "continuous",
     xlab = "Election Year",
     custom.labels = topic_labels,
     labeltype = "custom")
```

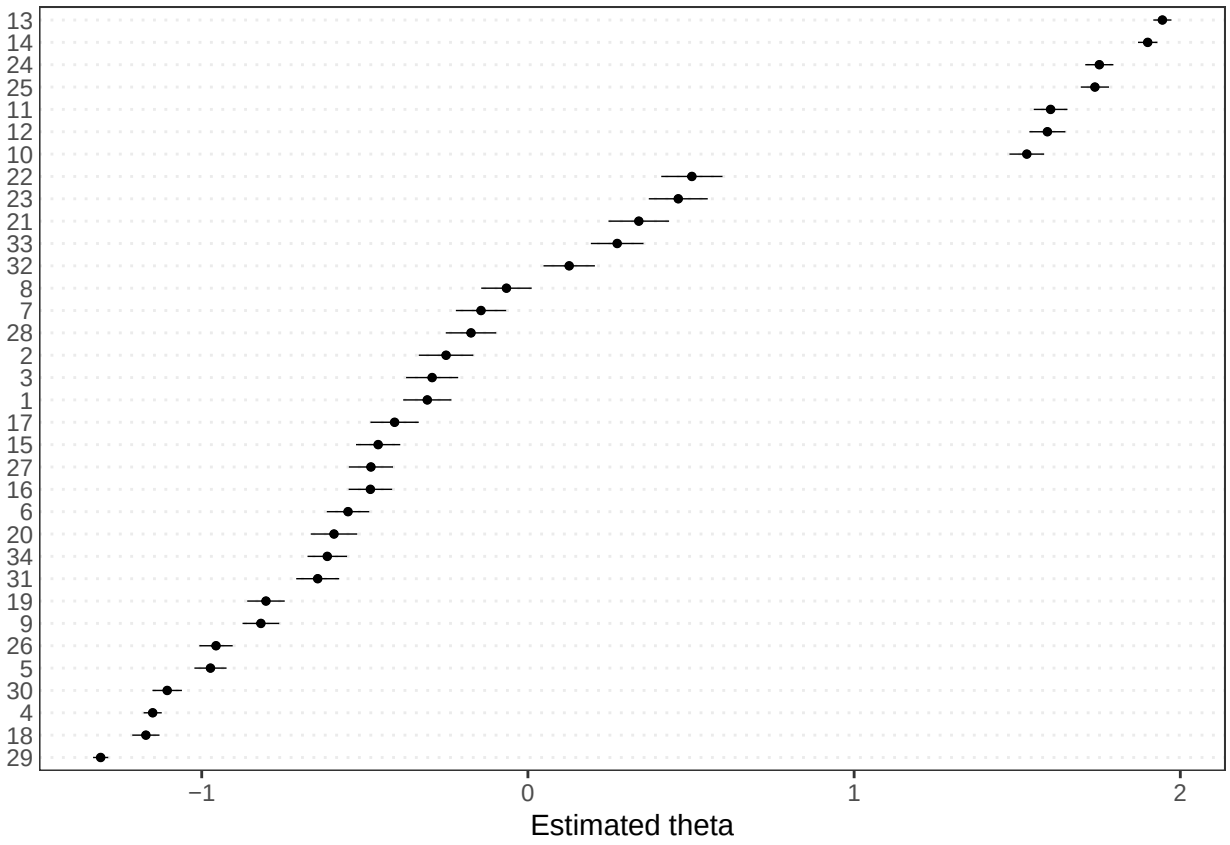


Wordfish

```
wfish <- textmodel_wordfish(dfm_debate_trim)
```

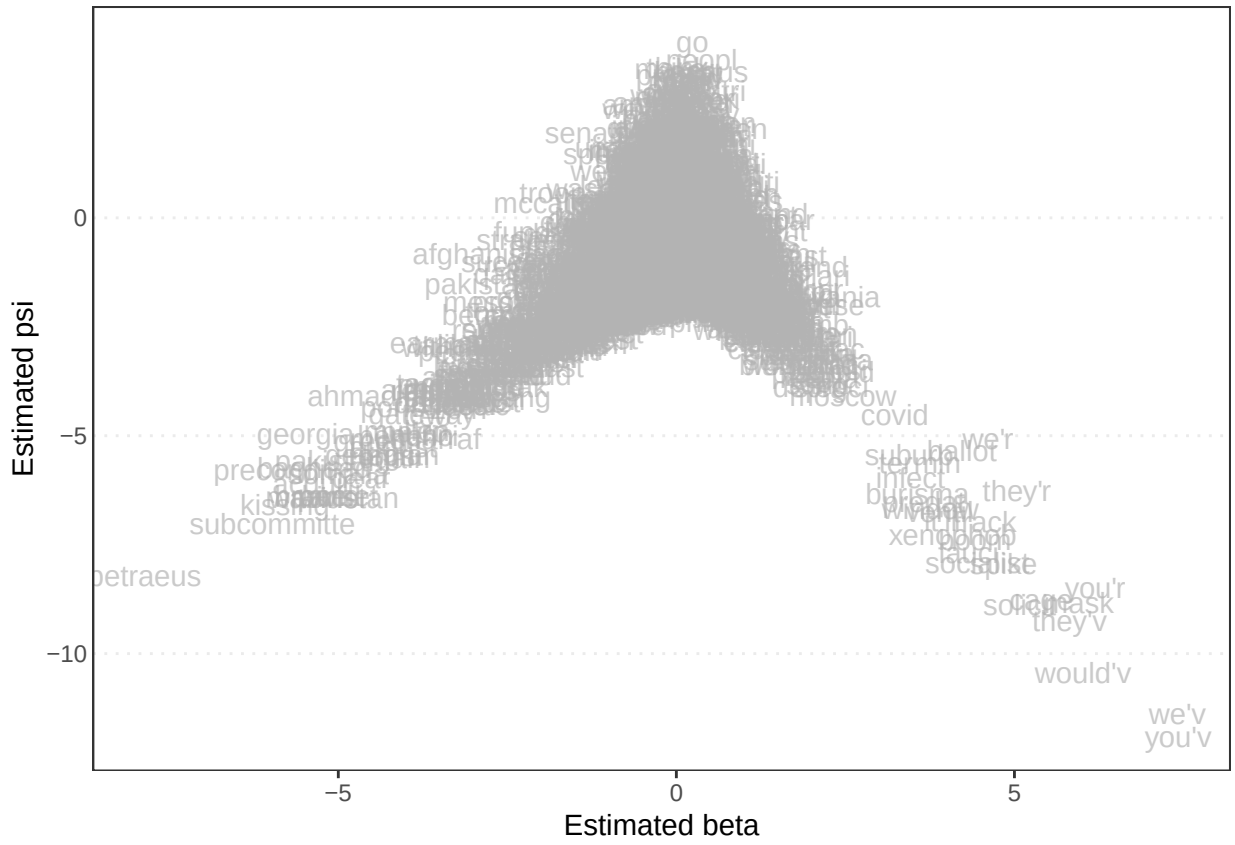
Plot Document Positions

```
textplot_scale1d(wfish, margin = "documents")
```



Plot Feature Positions

```
textplot_scale1d(wfish, margin = "features")
```



Save Theta Scores

```
docvars(dfm_debate_trim, "theta") <- wfish$theta
```

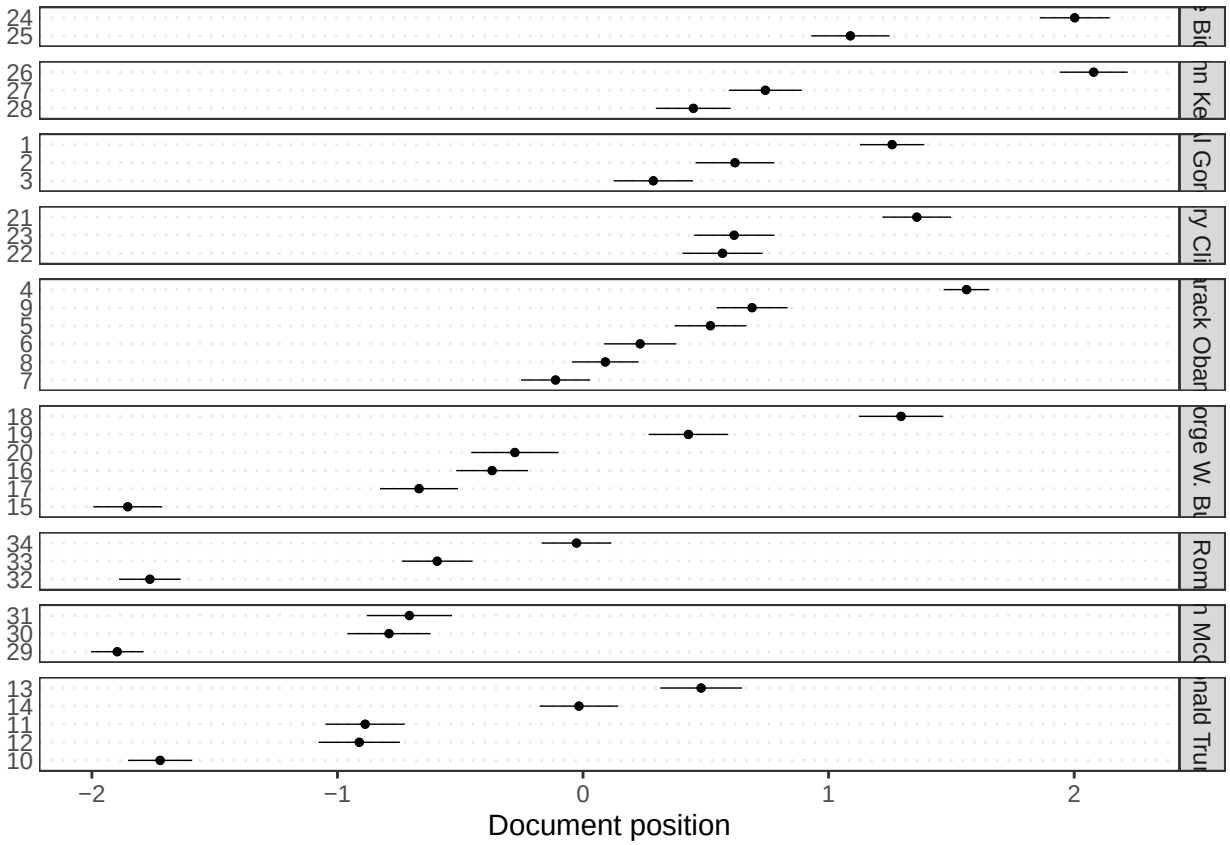
Wordscores

We pre-define one text by each Democrat candidate as `1` and for Republicans as `-1`.

```
dfm_debate_trim$refvalue <- c(1,rep(NA,2), #Al Gore
                              1,rep(NA,5), #Obama
                              -1,rep(NA,4),#Trump
                              -1,rep(NA,5),#Bush
                              1,rep(NA,2), #Hillary
                              1,NA,        #Biden
                              1,rep(NA,2), #Kerry
                              -1,rep(NA,2),#McCain
                              -1,rep(NA,2))#Romney

wscore <- textmodel_wordscores(dfm_debate_trim, dfm_debate_trim$refvalue)

lbg <- predict(wscore, se.fit = TRUE,
               interval = "confidence",
               rescaling = "lbg")
```

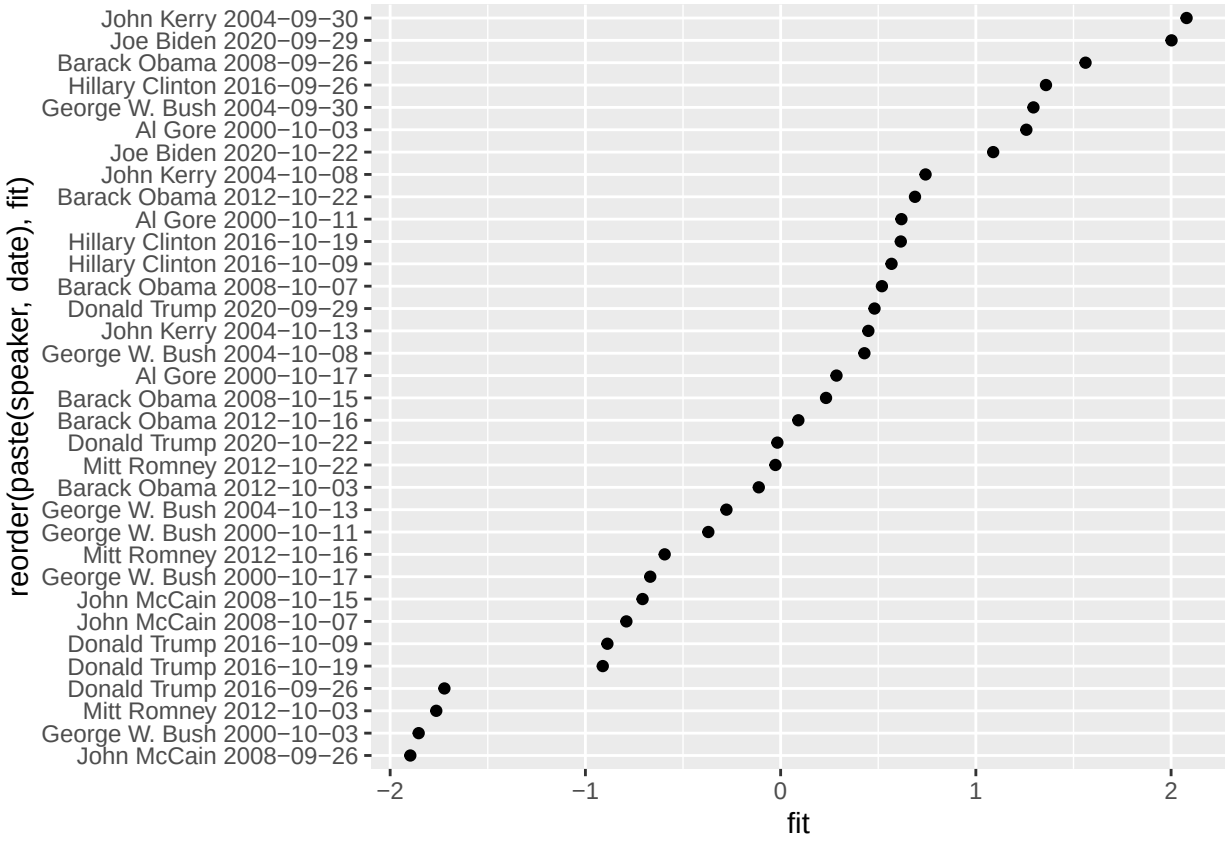
Visualise Scores per Text

```
textplot_scale1d(lbg,
                 margin = "documents",
                 groups = dfm_debate_trim$speaker)
```



Save and Plot Wordscores Positions

```
positions_ws <- lbg$fit %>% as.data.frame()

docvars(dfm_debate_trim) <- bind_cols(docvars(dfm_debate_trim),
                                      positions_ws)

ggplot(docvars(dfm_debate_trim),
       aes(x = fit,
           y = reorder(paste(speaker, date), fit))) +
  geom_point()
```



Thank you for your attention!